BST Implementation and Analysis Report with Experimental Results

By: Tania Nixon

Implementation

For the BST implementation, I used Gemini to provide the python code structure. All AI interactions are fully documented in another document. Overall, using AI was extremely beneficial for constructing this project. However, there were certain times where AI had limitations. One example is that after coding the three strategies separately, I asked for it to combine my three tables into one big table but it insisted on overhauling the code that I already had. Overall, I felt that the code it was outputting was convoluted. I also had an issue with downloading my AI interaction report and ended up copying and pasting all entries and outputs into a google document instead. However, a very useful aspect of AI in this project was its use in debugging. When I was implementing the code for the average ascending comparisons, I realized that I was receiving incorrect values. It pointed out that I was correct to think that and pointed out where and why my code was creating this issue.

Analysis

Analysis of the BST implementation will provide insight into divide and conquer efficiency. After completing the experiment of running the BST implementation with random seeds 14 to 24 using the random python package, we are able to compare heights, average node depths, and average search comparisons of the trees.

The experiment has shown that the insertion BST produces the tallest BST by a massive margin. This is because the keys are inserted in a strict sorted sequence and every subsequent key is larger than the preceding one. This forces the tree to only grow on the right as they only have right children. Since it never triggers the condition to add children on the left, the tree structure is essentially a single linked list and yields a height of $N - 1$ where N is the number of keys.

The height of the BST changes throughout the different strategies. For the ascending order BST changes linearly.

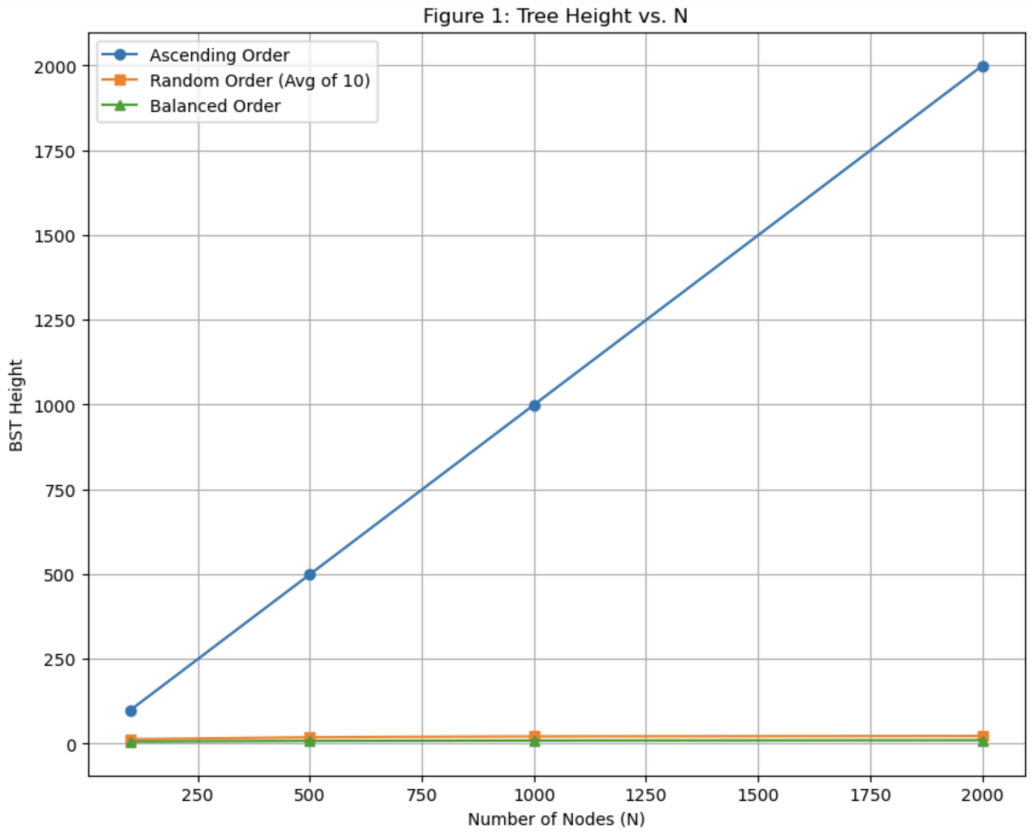


Figure 1: The tree height versus number of nodes

We can see this in Figure 1 that as N doubles, the height also doubles (if x = 500, then y = 500 also). This causes the ascending order BST to have the worst complexity outcome of $O(N)$ rather than $O(\log N)$. In contrast, the random order BST's height changes logarithmically because the shuffled elements ensure a good distribution of left and right children. This creates the average complexity outcome to be $O(\log N)$. Lastly, for the balanced order BST, the height of a perfectly balanced tree would be $log_2 n$. Using this method would give a worst case complexity outcome of $O(\log N)$.

In addition, the heights can also tell us about the search comparisons about these different strategies.

Size (N)	Strategy	Height	Avg Node Depth	Avg Search Comps
100	Ascending	99	49.50	51.5
	Random (Avg of 10)	12.8	6.57	7.8
	Balanced	6	4.80	5.8

500	Ascending Random (Avg of 10) Balanced	499 18.7 8	249.50 9.66 7.00	251.5 10.8 8
1000	Ascending Random (Avg of 10) Balanced	999 21.3 9	499.50 11.10 7.99	501.5 12 9
2000	Ascending Random (Avg of 10) Balanced	1999 22.6 10	999.50 12.02 8.98	1001.5 13.3 10

Table 1: BST experiment showing the height, average node depth, and average search comparison for all three strategies.

Looking at Table 1, the average search comparisons for the ascending and random insertion methods for the binary search tree are roughly half the value of the height of the tree. However, for the balanced insertion method the average search comparisons are almost equal to the height of the tree. Height is not the only indicator of performance for BSTs. For example, if Tree A were to have long, linear chains stretching to depth 20 with almost all 1000 nodes within the bottom, deeper layer its performance would be poor. Tree B on the other hand is almost perfectly balanced however it has one abnormally long branch that delves into depth 20. Comparing Tree A and B, we can see that tree height only reveals the depth of the deepest node but average search performance is determined by how all nodes are distributed across the levels of the tree.

According to the experiments, a randomly inserted BST is not guaranteed to be balanced. According to Table 1, the random insertion shows that the strategy produces a reasonably balanced tree on *average* of ten trials. However, whether the random insertion actually follows this behavior is random. Since the strategy is entirely reliant on the random picking of values, there is nothing stopping it from producing lopsided subtrees. This will result in poor run times just like the ascending order strategy.

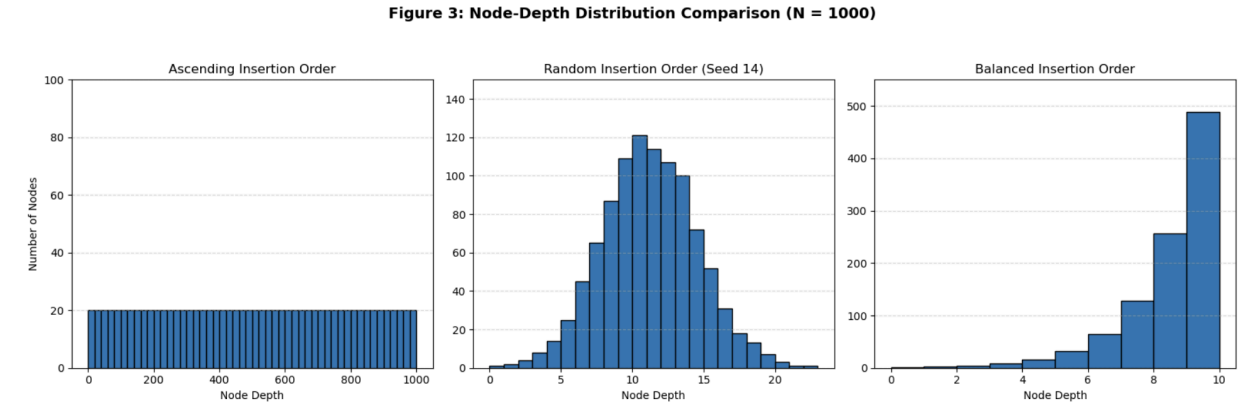


Figure 2: Node Depth Distribution Comparison

From our experiments we can also see how the node depths differ between the three insertion strategies. For ascending order, the strategy displays a perfect uniform distribution from the depth 0 to N-1. This is because the tree forms a single straight line where exactly one node is at every single depth level since the list is consecutive increasing numbers. From Figure 2 we can see that the distribution is completely flat. However, the random order strategy showcases a bell shaped normal distribution which can be seen in Figure 2. Finally, the balanced order strategy showcases a significant left leaning skew. That is because at N = 1000, about 75% of the dataset resides at depths 8 and 9 which we can see according to Figure 4. Additionally, From Table 1, we can see that there is a relationship between the average node depth and average successful search comparisons. The number of key comparisons will always be one greater than the node depth. This relationship can be expressed by: *Comparisons = Depth + 1*. We can see why this relationship exists if we look at the root of the tree. The root node is located at depth 0, yet it requires one value comparison to evaluate. Therefore, if every distinct key in a tree of size N is searched exactly once, the average search count is the average of all node depths added by one.

Appendix (Experimental Results)

Figure 1: The tree height versus number of nodes

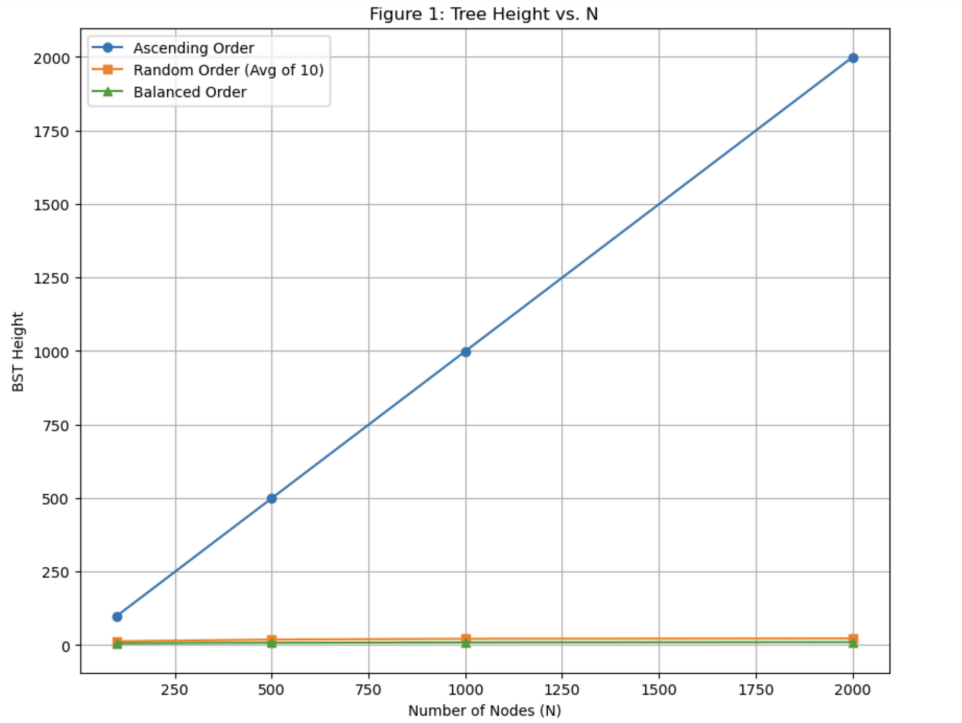


Figure 2: The average search comparisons versus nodes

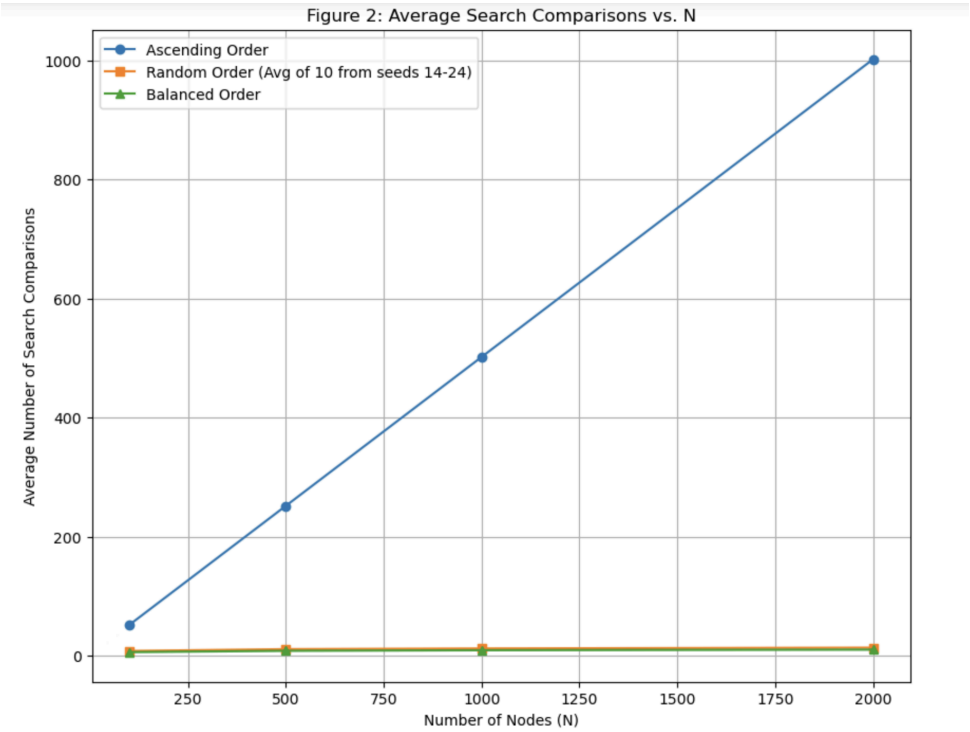


Figure 3: Node to depth distribution comparison of the three strategies

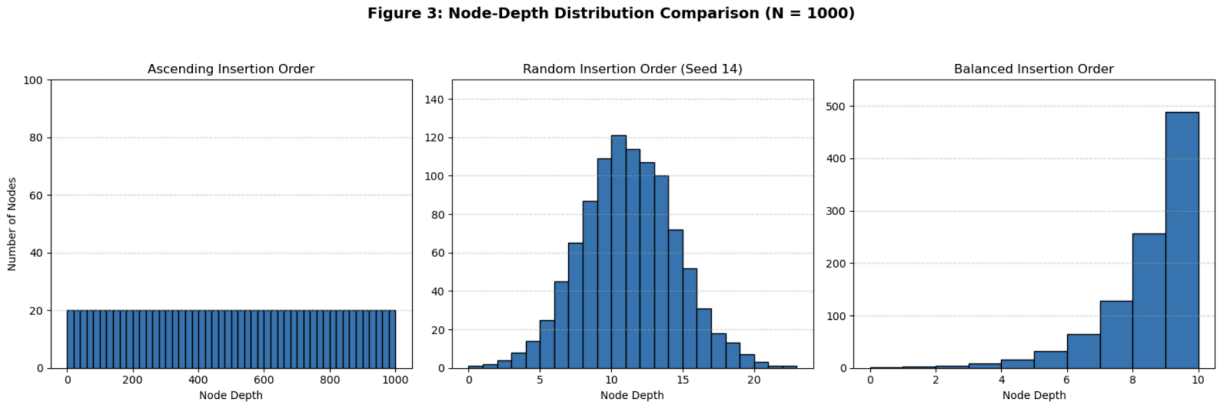


Figure 3.1: Node to depth distribution comparison of ascending insertion order BST

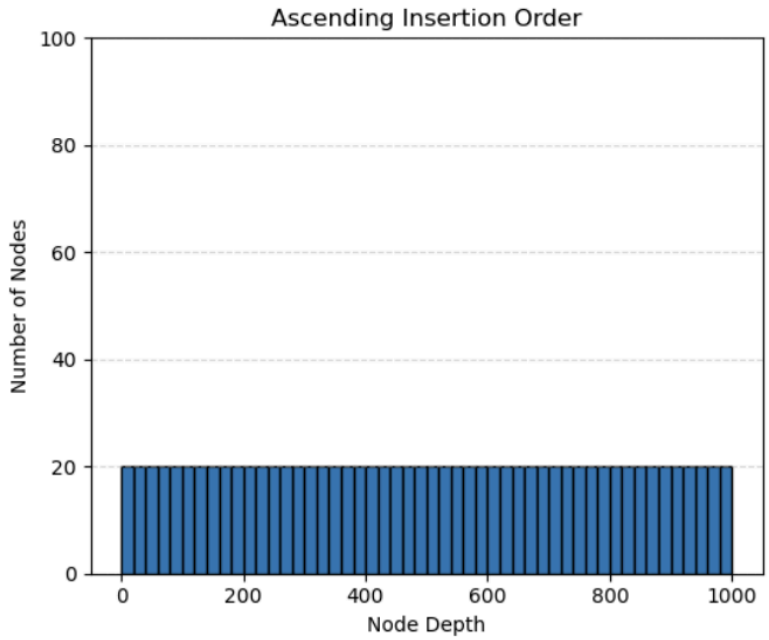


Figure 3.2: Node to depth distribution comparison of random insertion order BST

e 3: Node-Depth Distribution Comparison (N = 10

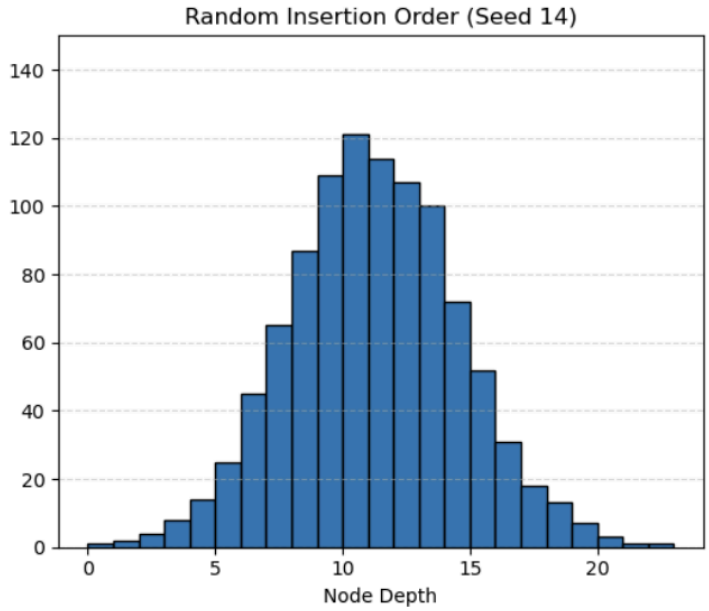


Figure 3.3: Node to depth distribution comparison of balanced insertion order BST

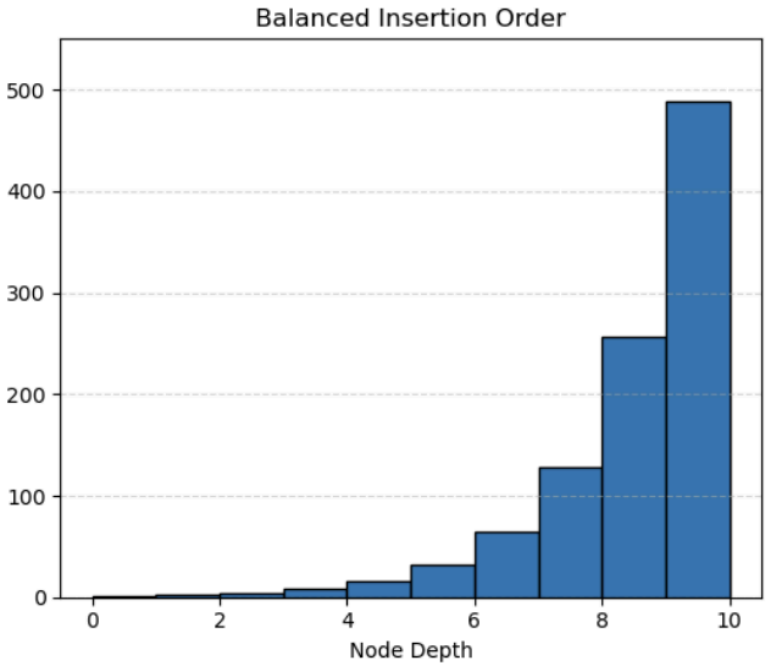


Table 1:

Size (N)	Strategy	Height	Avg Node Depth	Avg Search Comps
100	Ascending	99	49.50	51.5
	Random (Avg of 10)	12.8	6.57	7.8
	Balanced	6	4.80	5.8
500	Ascending	499	249.50	251.5
	Random (Avg of 10)	18.7	9.66	10.8
	Balanced	8	7.00	8
1000	Ascending	999	499.50	501.5
	Random (Avg of 10)	21.3	11.10	12
	Balanced	9	7.99	9
2000	Ascending	1999	999.50	1001.5
	Random (Avg of 10)	22.6	12.02	13.3
	Balanced	10	8.98	10